



TEAM PARADUCKS

FUTURE ENGINEERS

TABLE OF CONTENT

Content	Page Number
Who are we?	3 - 4
Mobility Manangement	5 - 7
Power and Sense Management	8 - 13
Obstacle Management	14 - 15
Programming	16 - 24
Gallery	25-27



MEET TEAM PARADUCKS

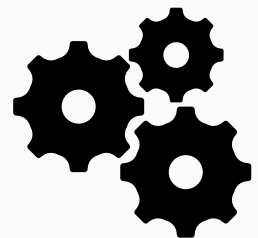
We are Team Paraducks from Bombay International School, a group of Grade 10 students competing in the WRO Future Engineers category. Driven by our passion for STEM and innovation, we aim to design creative, efficient, and intelligent robotic solutions that push the boundaries of technology and teamwork.



Team Paraducks is based in
Mumbai, India



Second-place finish at India
Nationals 2025



Our team is mentored by
The Innovation Story

FUTURE ENGINEERS



MEET THE TEAM

Neel



Neel Shah is a 10th grader at the Bombay International School and an aspiring engineer. He enjoys building robots, starting all the way from childhood with lego. His hobbies include sports such as squash and football, as well as researching and learning about new advances in STEM. He helps WRO NGO teams to improve their robot during their iteration phases.

Dev



Dev Shah is a passionate builder who loves STEM. He's a 10th grader and a student council member at the Bombay International School. He is also an avid debater, regularly participating in MUNs, and enjoys playing chess and golf in his free time. He mentors WRO teams to improve their overall robot in the final phases and helps for the notebook and interview

Tara

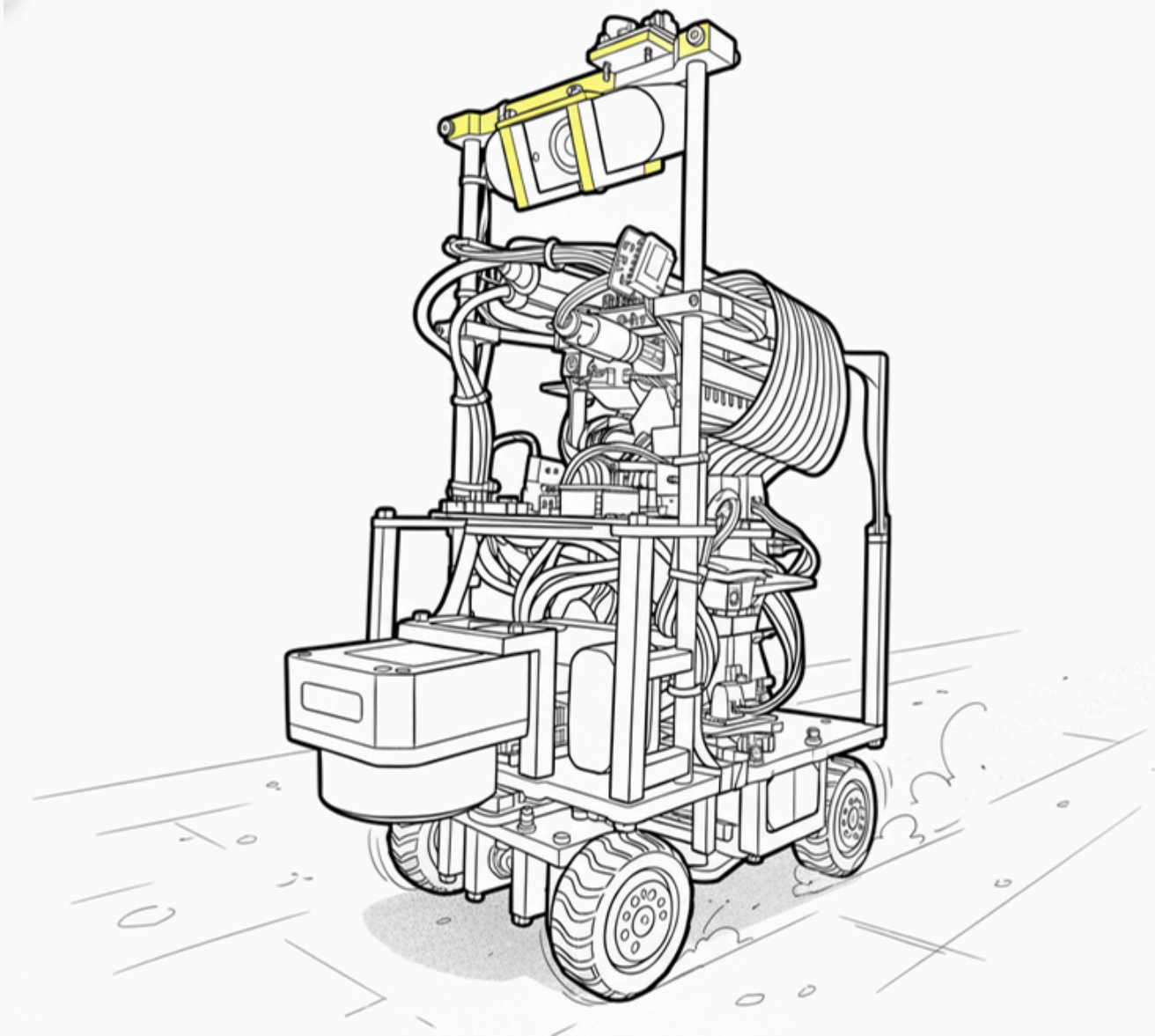


Tara Guha Thakurta is a 10th grader at the Bombay International School and a robotics engineer. She enjoys building robots and creating AI models for anything she likes. Her other hobbies are sports as she is an avid football fan. She aids WRO NGO teams in the software aspect during the preparation time.

FUTURE ENGINEERS



MOBILITY MANAGEMENT



FUTURE ENGINEERS



MOBILITY MANAGEMENT

DIGITAL SERVO



- Front axle servo gives up to 90° steering.
- Free rear axle lets wheels self-align while driving.
- Disjoint shaft minimizes slip by keeping wheels aligned with the steering.

DC MOTOR



- 12V Rhino DC geared motor with encoder is reliable and efficient.
- Provides steady torque for moderate speeds.
- Drives the rear axle via a bevel gearbox.
- Powered through a motor driver from the battery PCB.

GENERAL



- Servo steers the front axle up to 90°.
- Free rear axle lets wheels self-align.
- Disjoint shaft reduces wheel slip.

FUTURE ENGINEERS



MOBILITY MANAGEMENT

MOTOR DRIVER



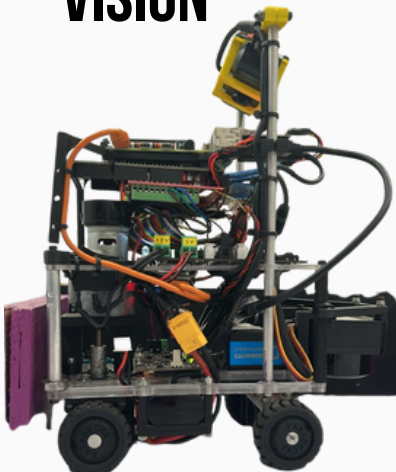
- Cytron MD10C R3 drives the brushed DC motor.
- Delivers efficient continuous current for stable performance.
- Supports smooth bidirectional control.
- Built-in overcurrent and thermal protection for reliability.

- Differential lets wheels rotate at different speeds for smooth turns.
- Reduces slip by evenly distributing torque.
- Enables ~10 cm turning radius for tight maneuvers.
- Improves traction and stability during turns.

DIFFERENTIAL GEAR



VISION

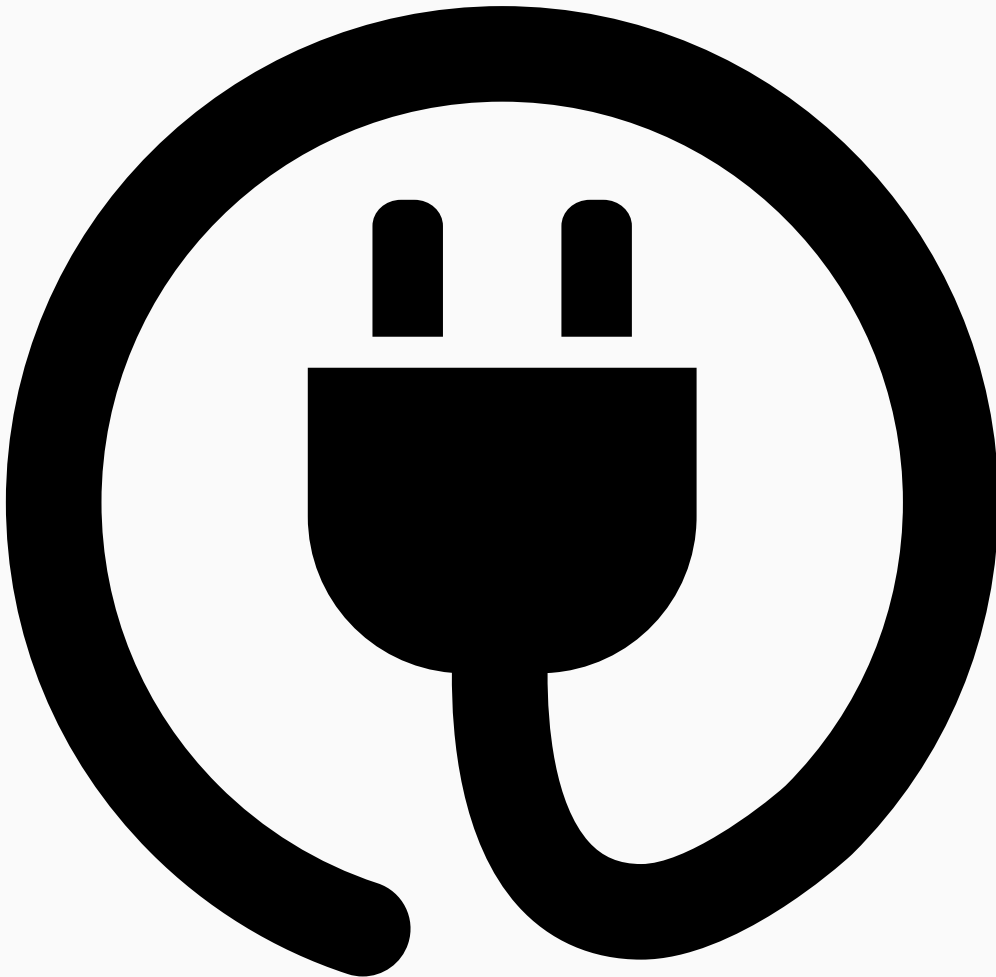


- Camera mounted 11" high at a 20° tilt for wider block detection.
- Front C1 LiDAR enhances awareness for faster, precise turns.
- TF Mini LiDARs (front, left, right) assist in obstacle detection and steer control.

FUTURE ENGINEERS



POWER MANAGEMENT AND SENSOR INTEGRATION



FUTURE ENGINEERS

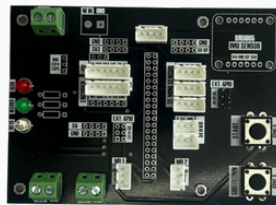


POWER

Power Supply: Pro-Range LiPo 11.1 V 1300mAh 3-cell battery



The motor driver is powered by the **custom PCB** to run the motor.



A **12V-5V buck converter** powers all sensors via the custom PCB



- **Raspberry Pi 4B + Coral** powered through a **buck converter** (USB-A to USB-C).
- Coral TPU accelerates model processing and boosts FPS.

- **Arduino MEGA** is powered from the Raspberry Pi's USB-A to USB-B connection.
- The same USB link is also used for serial communication between both devices.

FUTURE ENGINEERS



COMPONENTS

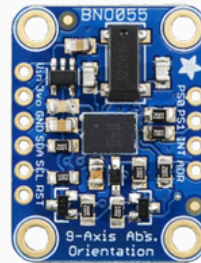
Sensors



RPLidar C1



Hikvision Camera

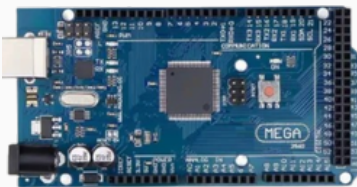


BNO085 IMU



TFmini LiDAR

Controlling unit



Arduino Mega 2560



MD10C Motor Driver

Power



LIPO
12V Battery



Switch

Processing unit



Rasberry Pi 4B



Google Coral

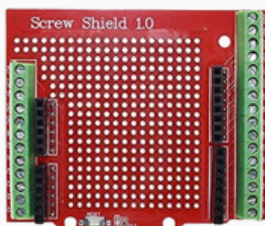
Actuators



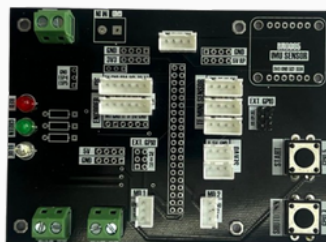
DC Motor



RDS3235
35kg Servo



Screw Shield



Custom PCB

Power management



REES52 12V-5V Converter

CIRCUIT

Hikvision Camera



Key:

-  = Ground
-  = Power
-  = Data



RPLidar C1

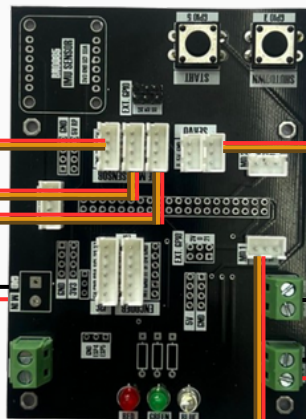


RasPi 4B



Google Coral

TFmini LiDAR



RDS3235 35kg
Servo

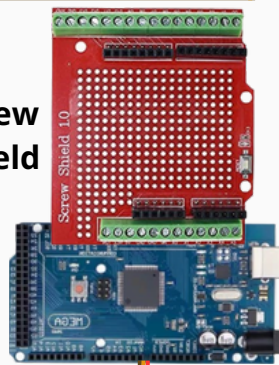
MD10C Motor
Driver



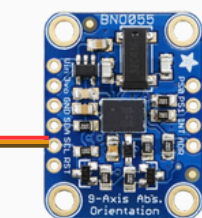
REES52 12V-5V
Converter



Screw
Shield



Arduino Mega
2560



BNO085 IMU

Switch



DC motor

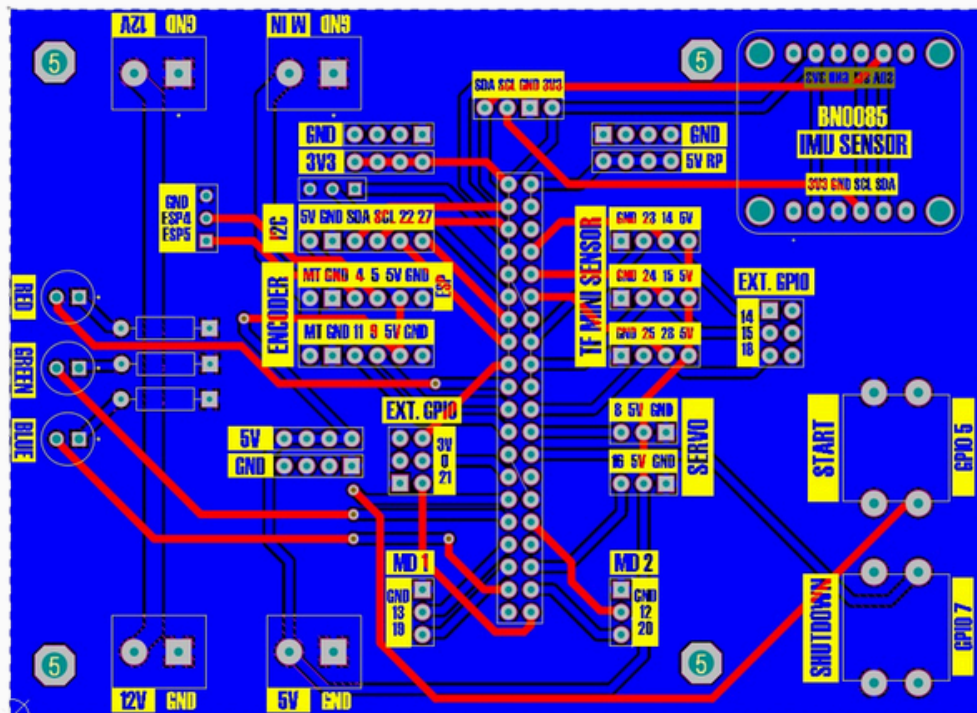
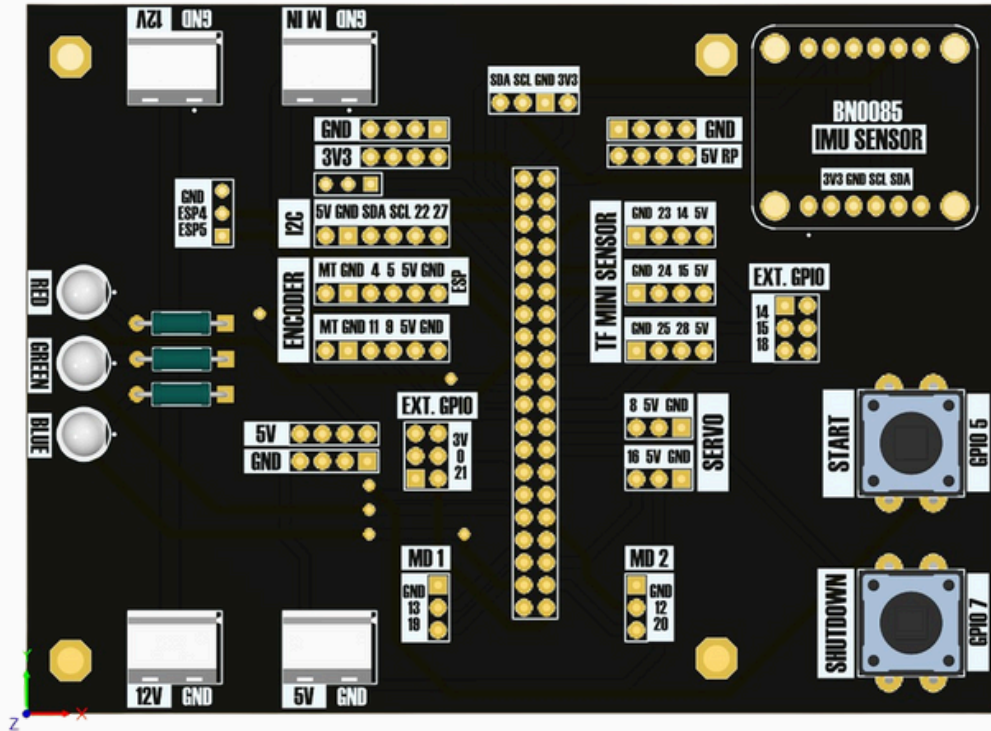


LIPO 12V Battery
1300mAh

FUTURE ENGINEERS



CUSTOM PCB



FUTURE ENGINEERS



MICROCONTROLLERS

EARLY PROTOTYPES

- Started with Arduino UNO + HuskyLens, but processing power was insufficient.
- Switched to ESP32 DevKit; better performance but faced compatibility and connectivity issues.
- These trials led us to the stable and more powerful setup used in the final robot.

CURRENT ROBOT

- Raspberry Pi 4B used as the main processor, running Python with better object detection and multiprocessing.
- Pi alone didn't provide enough FPS.
- Added Google Coral TPU to accelerate model processing and significantly increase FPS.



- Arduino MEGA 2560 Rev3 used as the secondary microcontroller.
- Chosen for its strong sensor compatibility and serial communication libraries.
- Handles motor encoder and IMU data.
- Communicates with the Raspberry Pi via UART.



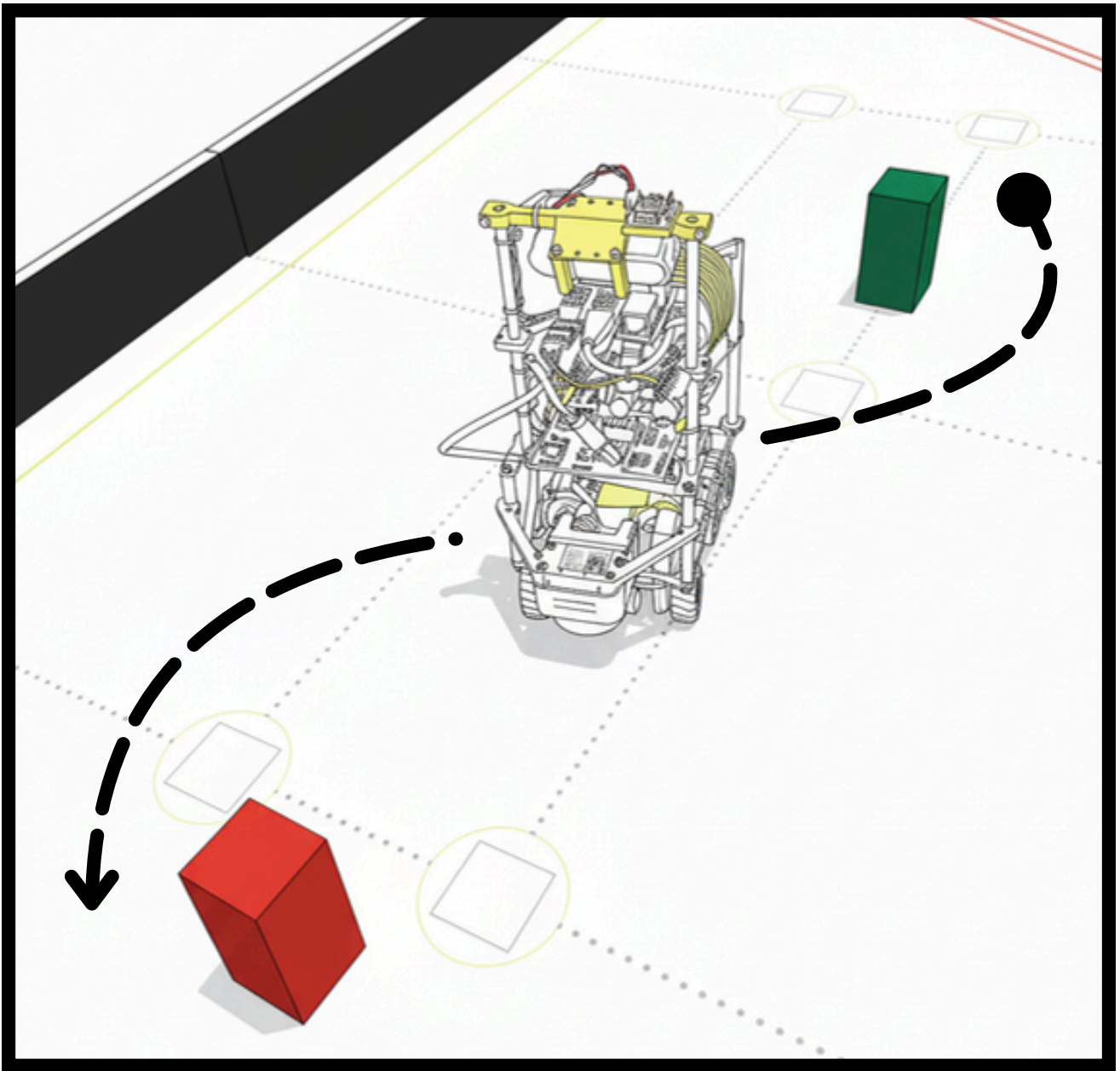
FUTURE ENGINEERS



SENSORS

Name	Description	Use
 C1 LiDAR	Compact laser sensor, measures distance with high precision, 360° Horizontal FOV	Mapping surroundings and for turning corners
 Hikvision Camera	High-resolution colour camera, supports video recording and streaming Horizontal FOV = 86° Vertical FOV = 55°	Object detection, visual navigation
 TF-mini LiDAR	Small and accurate LiDAR, up to 12 meters range	Detecting nearby objects, simple distance sensing
 QUAD Encoder	Rotary encoder, CPR at output shaft of motor	Monitoring robot movement and controlling speed
 IMU (BNO085)	Smart 9-axis sensor for comprehensive motion and orientation sensing	Tracking orientation, measuring acceleration

OBSTACLE MANAGEMENT



FUTURE ENGINEERS



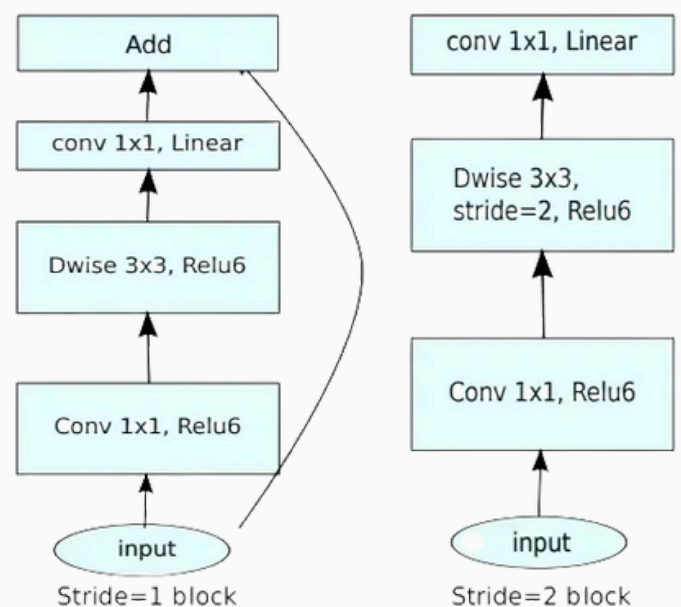
OBJECT DETECTION

MACHINE LEARNING MODEL

MobileNetSSDv2 (MobileNet Single Shot Detector) is an object detection model with 267 layers and 15 million parameters. It provides real-time inference under compute constraints in devices.

BENEFITS OF THE CURRENT SYSTEM

- **Efficient on Low-Power Devices:** Highly optimised for hardware like Raspberry Pi and microcontrollers, delivering accurate, fast object detection.
- **Real-Time Detection and Robustness:** Detects and classifies objects instantly, providing reliable results.
- **Easy Customization and Support:** Works with major Machine learning frameworks, allowing quick adaptation.
- **Compact and Fast:** Small model size with high performance



(d) Mobilenet V2

INPUT & OUTPUT SPECIFICATIONS

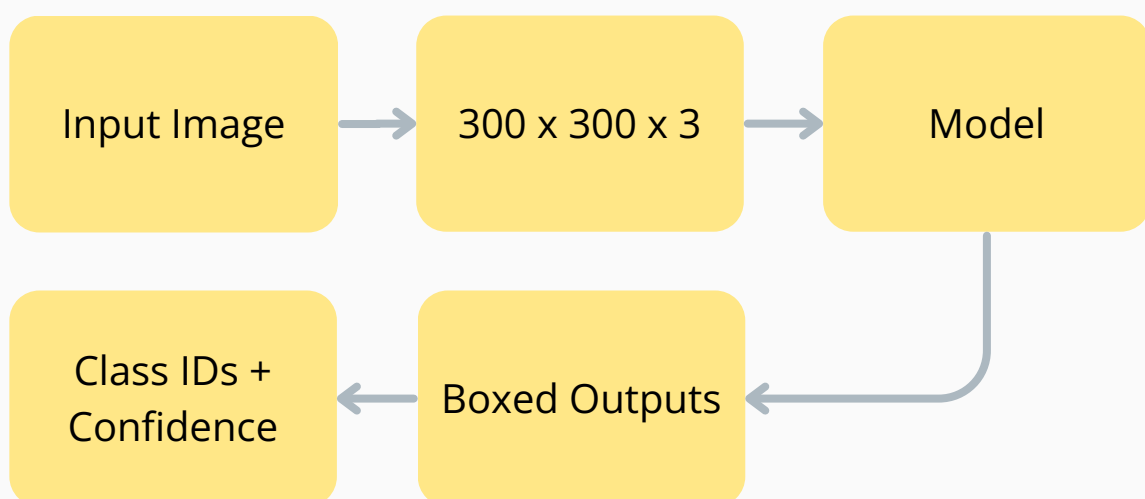
Input Format

- RGB image (300×300×3)
- Resized + normalized
- Single frame per inference

Output Format

- Bounding boxes (ymin, xmin, ymax, xmax)
- Class ID
- Confidence score
- Number of detections

I/O Diagram



FUTURE ENGINEERS

DATASET & TRAINING

Dataset Preparation (Roboflow)

A custom dataset was built using Roboflow: - **5,193 annotated** + augmented images - **Augmentations**: rotations, flips, brightness changes - Split: 70% train / 15% validation / 15% test

Training Pipeline

- Transfer learning from pretrained MobileNet-SSD v2
- 100 epochs
- Adam optimizer (LR 0.001)
- Classification + Localization losses

Training Flow



Training Results

- **mAP (mean absolute precision): 87%**
- Stable training/validation curves
- Good generalization across lighting conditions

FUTURE ENGINEERS

DEPLOYMENT & RESULTS

Deployment Setup

- Format: TFLite
- Hardware: Raspberry Pi 4 + Coral Edge TPU
- Inference Speed: **~15 FPS**

Deployment Diagram



On-Robot Performance

- Latency: 25–30 ms
- Range: 20–150 cm
- Real-time stable detections
- Works reliably under different lighting

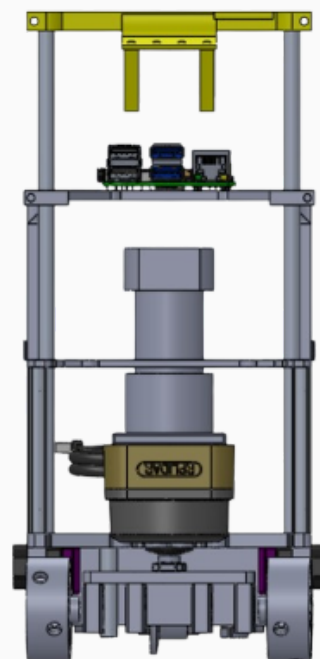
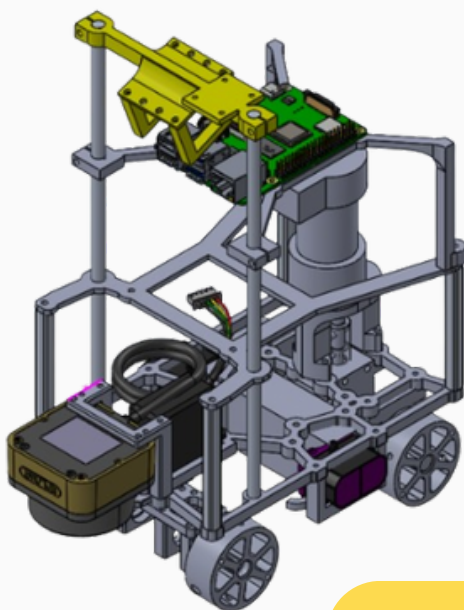
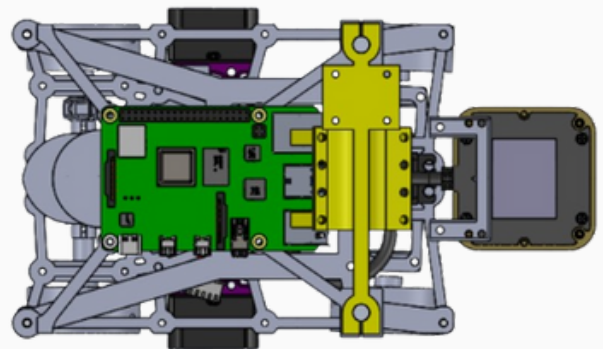
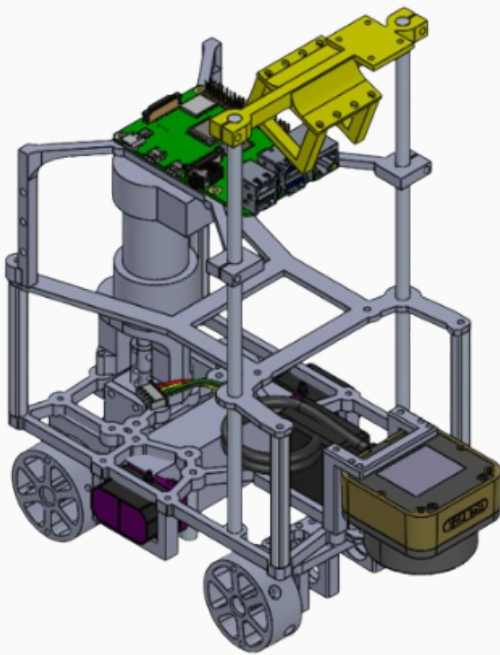
Future Improvements

- Add more diverse training conditions
- Apply temporal smoothing (Kalman filter)
- Try next-gen lightweight models (YOLO-N, MobileNet-SSD v3)

FUTURE ENGINEERS

CAD

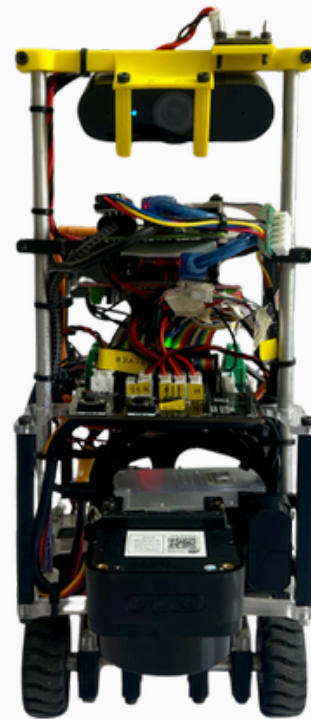
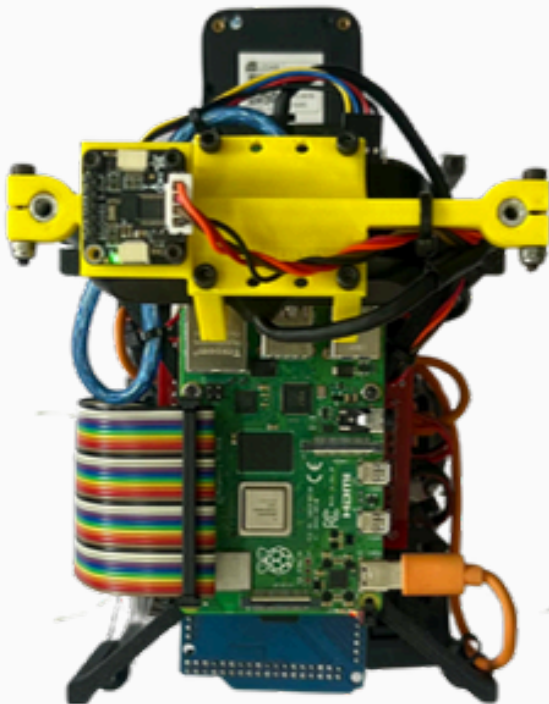
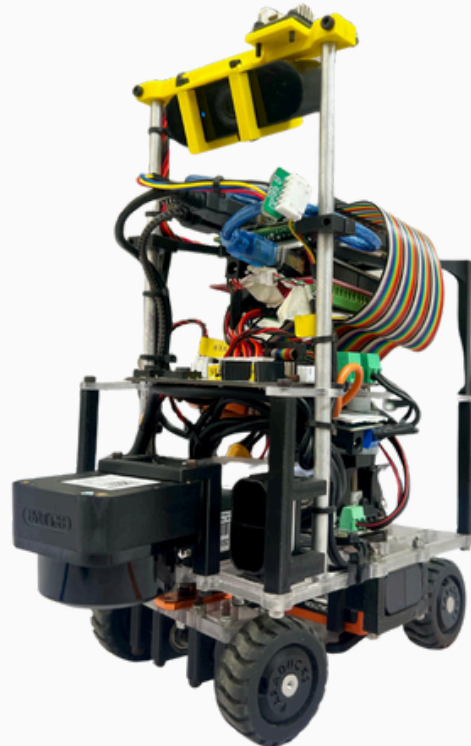
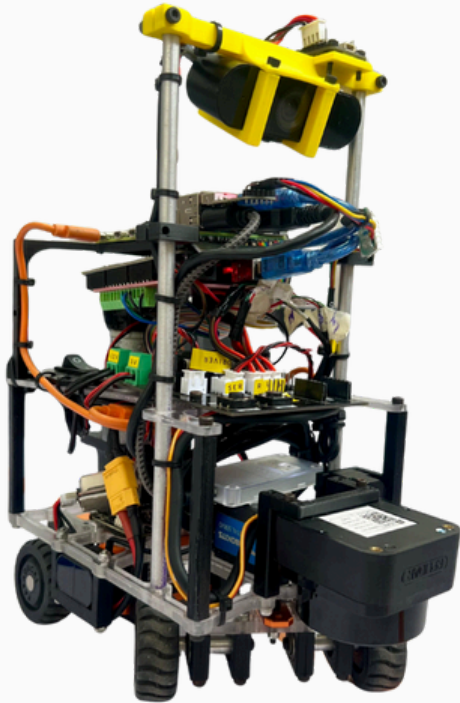
The robot's digital model was developed in SolidWorks, enabling precise 3D CAD design for accurate component sizing and custom 3D-printed mounts, ensuring compliance with strict robot size constraints.



FUTURE ENGINEERS



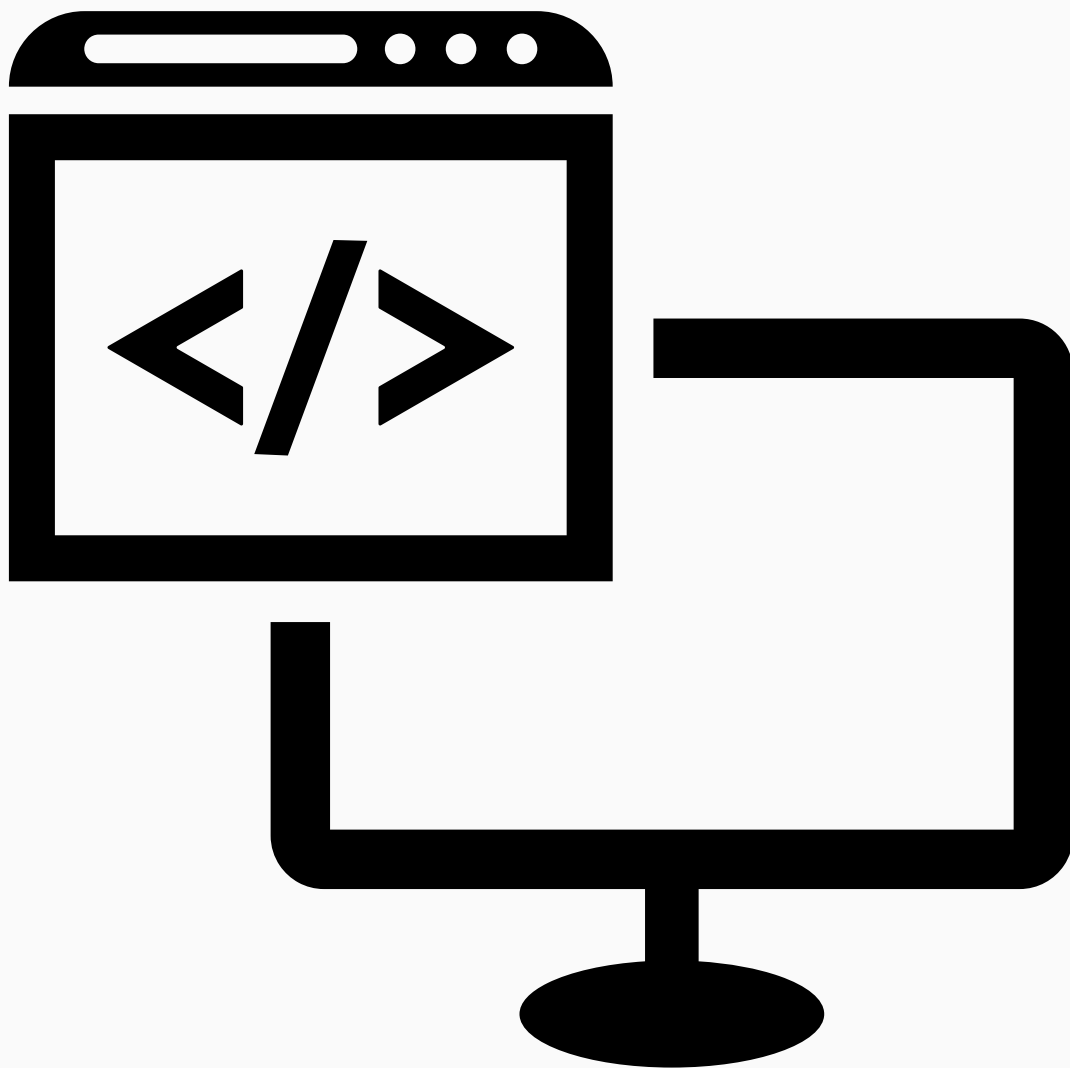
PHOTOS



FUTURE ENGINEERS



PROGRAMMING



FUTURE ENGINEERS



PROGRAMMING

Key Functions:

TFMini Data: Calculates the distance from the nearest obstacle to itself

Set Angle: Uses PWM (pulse-width modulation) to set the angle, converts degree input to pulse-width

C1 lidar: Has 360 degree view and is able to read distance from each angle to wall, we use this for turning

Running Encoder: Establishes UART communication with Arduino MEGA: reads, decodes, and strips lines of data received

PID controller: Used to decrease error between desired and current position of servo

Programming Languages:

- Python
- C

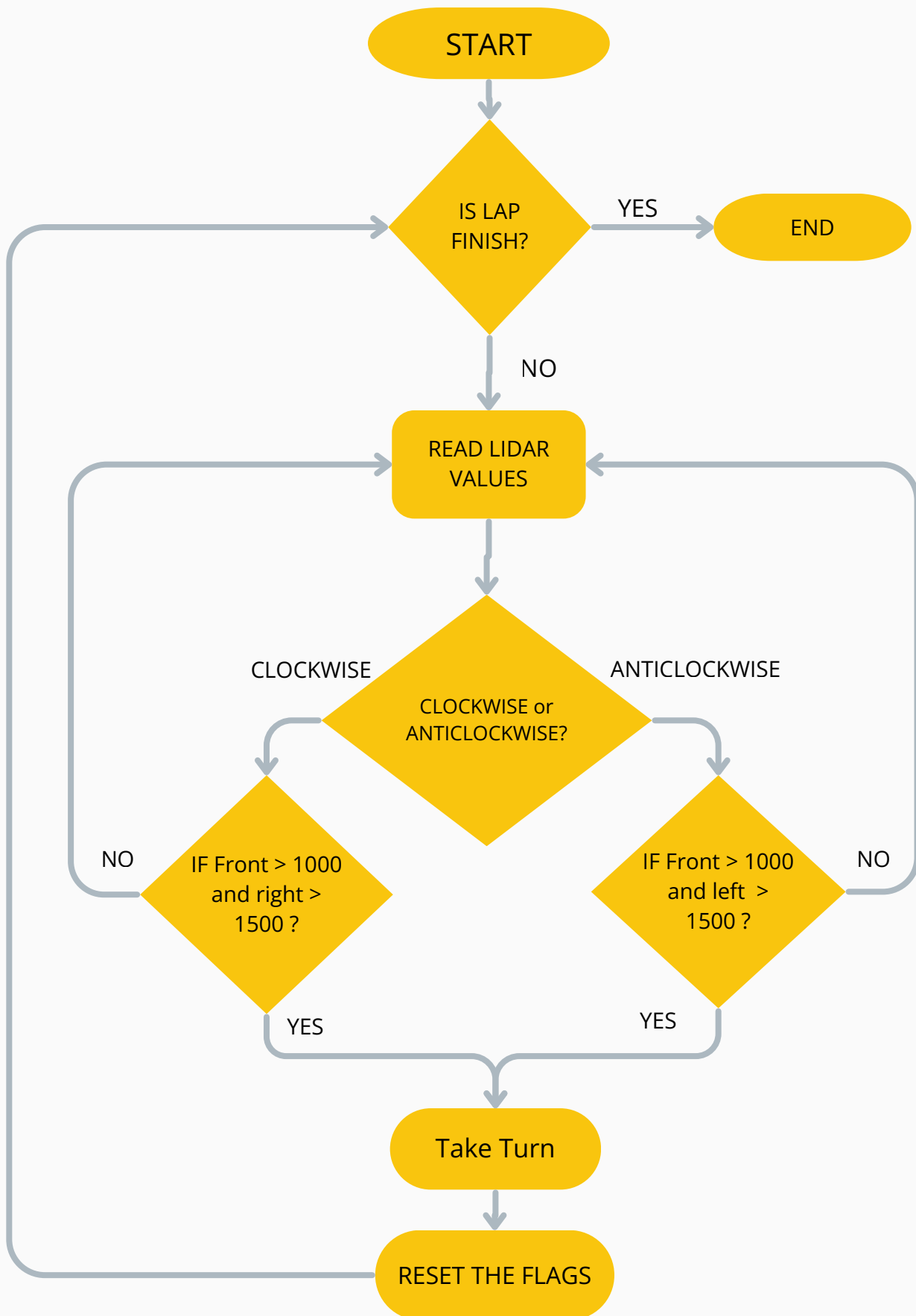
Turning Logic:

The turning logic is based on LIDAR sensor readings and special direction flags:

If there is an obstacle close in front ($\text{lidar_front} < 900$), and space to the right ($\text{lidar_right} > 1500$), and the right flag is set (right_f.value) but not the left flag, the robot triggers a right turn ($\text{turn_trigger.value} = \text{True}$).

If there is an obstacle close in front, and space to the left ($\text{lidar_left} > 1500$), and the left flag is set (left_f.value) but not the right flag, the robot triggers a left turn. If neither condition is met, the robot does not trigger a turn ($\text{turn_trigger.value} = \text{False}$).

TURNING FLOWCHART



PSEUDOCODE

TURNING

```
IF sp_angle > 180 THEN #sp_angle is a setpoint angle
    sp_angle ← sp_angle - 360
ENDIF
IF previous_angle ≠ angle THEN
    IF prev_sp ≠ sp_angle THEN
        sp_angle ← 360 - sp_angle
    ENDIF
    prev_sp ← sp_angle
    WHILE (angle - previous_angle) > 1 DO
        lidar_angle ← (previous_angle + 1) MOD 360
        lidar_distance ← previous_distance
        previous_angle ← lidar_angle
    ENDIF
    IF lidar_angle = ((0 + imu_r + sp_angle) MOD 360) THEN
        lidar_front ← lidar_distance
        lidar_f ← lidar_front
    ENDIF
    IF lidar_angle = ((90 + imu_r + sp_angle) MOD 360) THEN
        lidar_left ← lidar_distance
    ENDIF
    IF lidar_angle = ((270 + imu_r + sp_angle) MOD 360) THEN
        lidar_right ← lidar_distance
    ENDIF
    # Turning decision based on lidar readings and flags
    IF (lidar_front < 900 AND lidar_right > 1500) AND right_flag = TRUE AND left_flag = FALSE THEN
        right_turn_trigger ← TRUE
    ELSE IF (lidar_front < 900 AND lidar_left > 1500) AND left_flag = TRUE AND right_flag = FALSE THEN
        left_turn_trigger ← TRUE
    ELSE
        right_turn_trigger, left_turn_trigger ← FALSE
    ENDIF
```

PSEUDOCODE

OPEN CHALLENGE

```
INIT hardware, reset Arduino, open serial, set variables

PROCEDURE CorrectAngle(Target, Heading, TF_L, TF_R)
  error ← Heading - Target
  IF error > 180 THEN
    error ← error - 360
  ENDIF
  correction ← kp*error + kd*(error-prevError) + ki*(totalError+error)

  IF TF_L < 15 THEN
    correction ← correction - 20
  ELSE IF
    TF_R < 15 THEN correction ← correction + 20
  ENDIF

  LIMIT correction TO -30..30
  SetServo(90 + correction)
ENDPROCEDURE

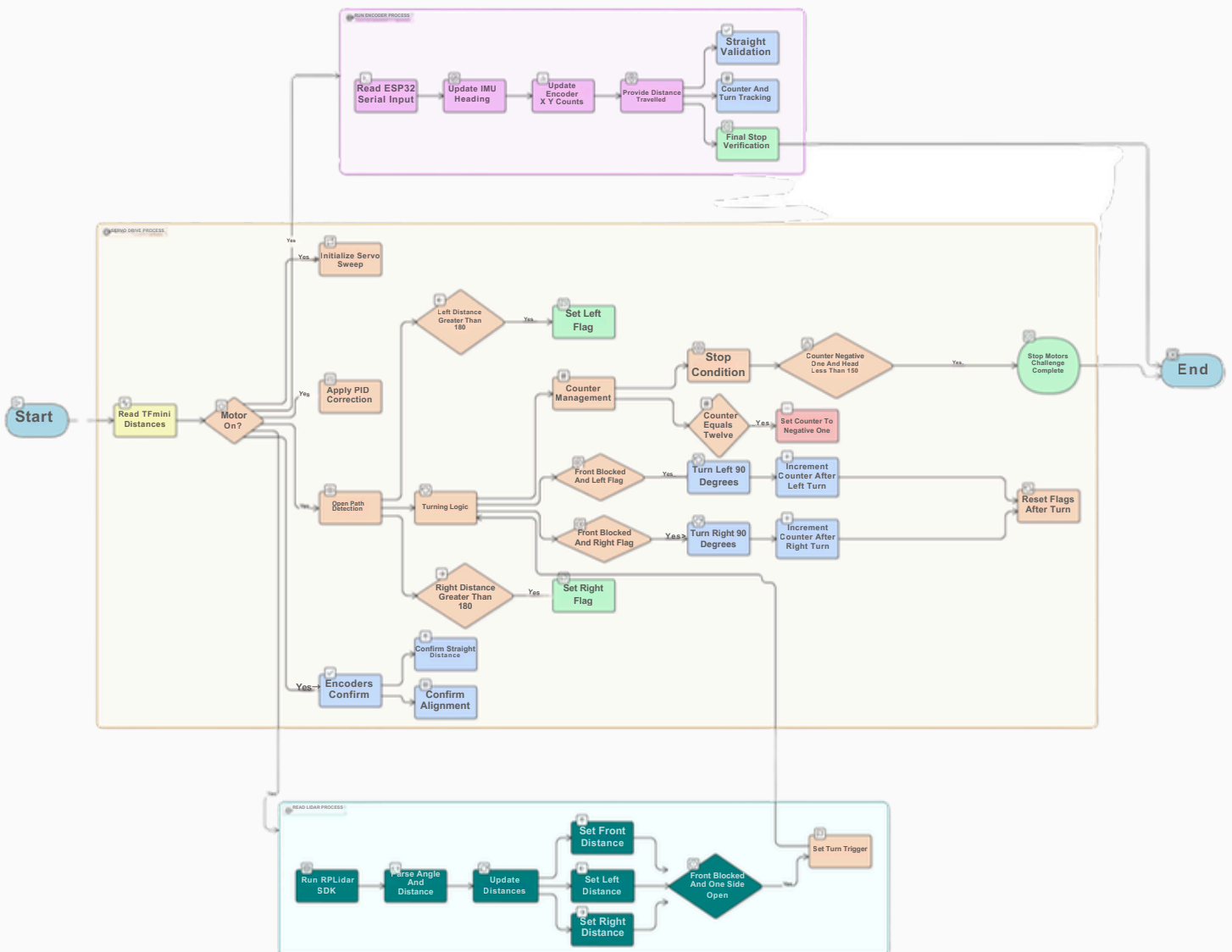
PROCEDURE ServoDrive()
  WAIT button
  LOOP
    READ Heading, TF_L, TF_R
    IF button pressed THEN
      ENABLE motor
      IF no flags THEN
        IF TF_R > 180 THEN
          RightFlag ← TRUE
        ELSE IF TF_L > 180 THEN
          LeftFlag ← TRUE
        ENDIF
      ENDIF
      CALL CorrectAngle(Target, Heading, TF_L, TF_R)
      IF RightFlag AND right > 130 AND front < 75 AND NOT Trigger THEN
        counter ← counter+1
        Target ← (90*counter) MOD 360
        Trigger ← TRUE
      ELSE IF LeftFlag AND left > 130 AND front < 75 AND NOT Trigger THEN
        counter ← counter+1
        Target ← -(90*counter) MOD 360
        Trigger ← TRUE
      ELSE IF (right < 85 OR left < 85) AND front > 75 THEN
        Trigger ← FALSE
      ENDIF
    ELSE
      STOP motor, RESET flags
      CALL CorrectAngle(0, Heading, TF_L, TF_R)
    ENDIF
  ENDLOOP
ENDPROCEDURE

PROCEDURE RunEncoder()
  LOOP READ serial › update Heading, EncoderCounts
  ENDLOOP
ENDPROCEDURE

PROCEDURE ReadLidar()
  LOOP
```

FLOWCHART

OPEN CHALLENGE



FUTURE ENGINEERS



PSEUDOCODE

OBSTACLE CHALLENGE

Detection and block avoidance

```
PROCEDURE DriveLogic:
  IF DetectGreenBlock() THEN
    power ← 70
    CALL CorrectPosition(setPointL, heading_angle, x, y, counter, blue_flag, orange_flag)
  ELSEIF DetectRedBlock() THEN
    power ← 70
    CALL CorrectPosition(setPointR, heading_angle, x, y, counter, blue_flag, orange_flag)
  ELSEIF DetectPinkBlock() THEN
    IF blue_flag THEN
      IF TF_Left < 30 THEN
        CALL CorrectAngle(heading_angle + 10, CurrentHeadValue())
      ELSE
        CALL CorrectAngle(heading_angle, CurrentHeadValue())
      ENDIF
    ELSEIF orange_flag THEN
      IF TF_Right < 30 THEN
        CALL CorrectAngle(heading_angle - 10, CurrentHeadValue())
      ELSE
        CALL CorrectAngle(heading_angle, CurrentHeadValue())
      ENDIF
    ENDIF
  ELSE
    power ← 85
    CALL CorrectPosition(setPointC, heading_angle, x, y, counter, blue_flag, orange_flag)
  ENDIF
  CALL UpdateMotors(power)
ENDPROCEDURE
```


PSEUDOCODE

OBSTACLE CHALLENGE

System Starts

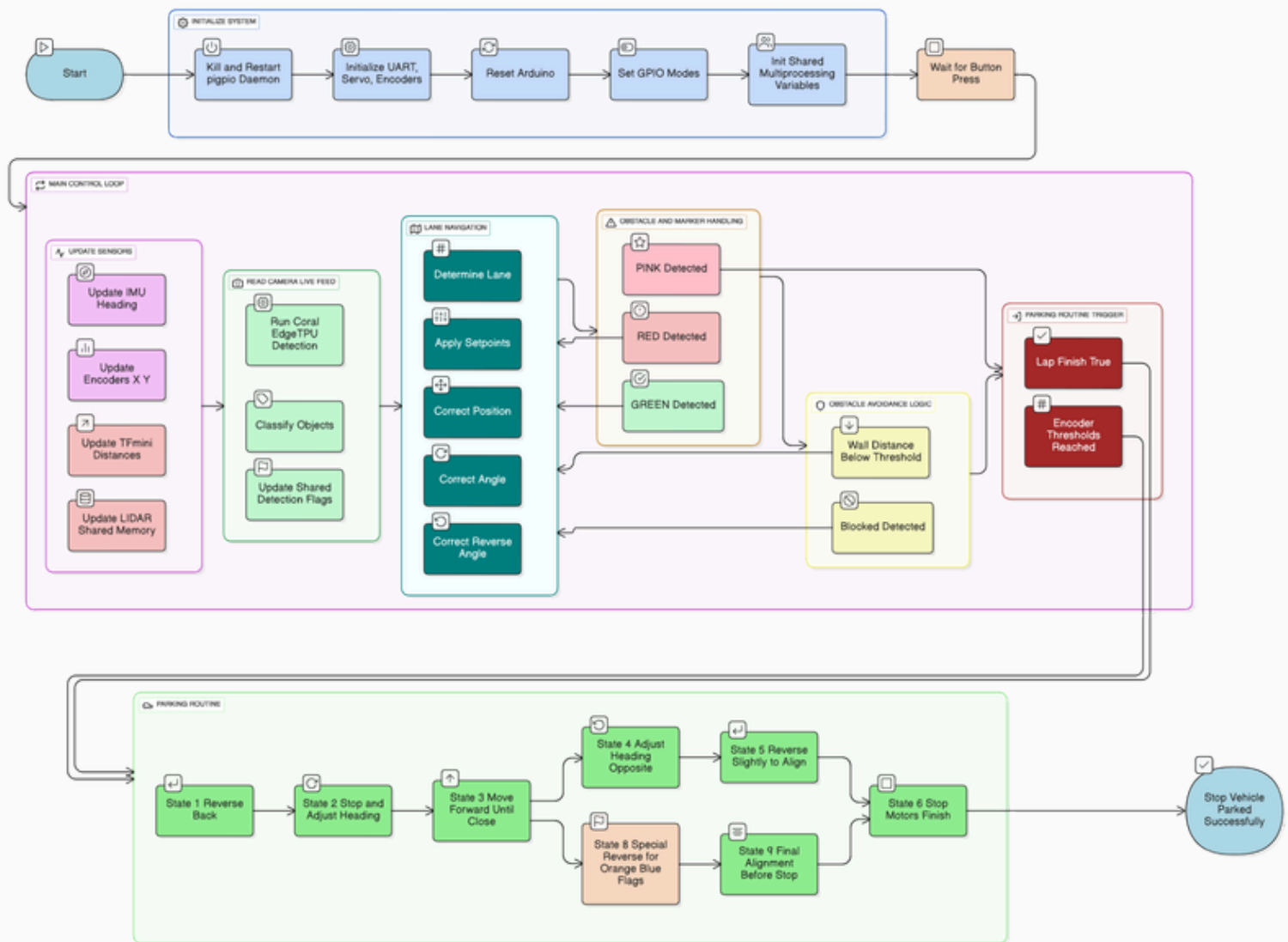
```
PROCEDURE Main
  INITIALISE all pins and devices
  RESET Arduino
  CREATE Servo using servo_pin
  CREATE TFinini using RX_Head, RX_Left, RX_Right, RX_Back, EncoderCounter
  inParkingAtStart ← FALSE
  lap_finish, continue_parking, blue_flag, orange_flag, parking_flag ← FALSE
  counter ← 0
  WHILE TRUE
    IF ButtonIsPressed() THEN
      DriveRoutine()
    ELSE
      StopMotors()
      heading_angle ← 0
      counter ← 0
      CALL CorrectAngle(heading_angle, CurrentHeadValue())
    ENDIF
  ENDWHILE
ENDPROCEDURE
```

Parking sequence

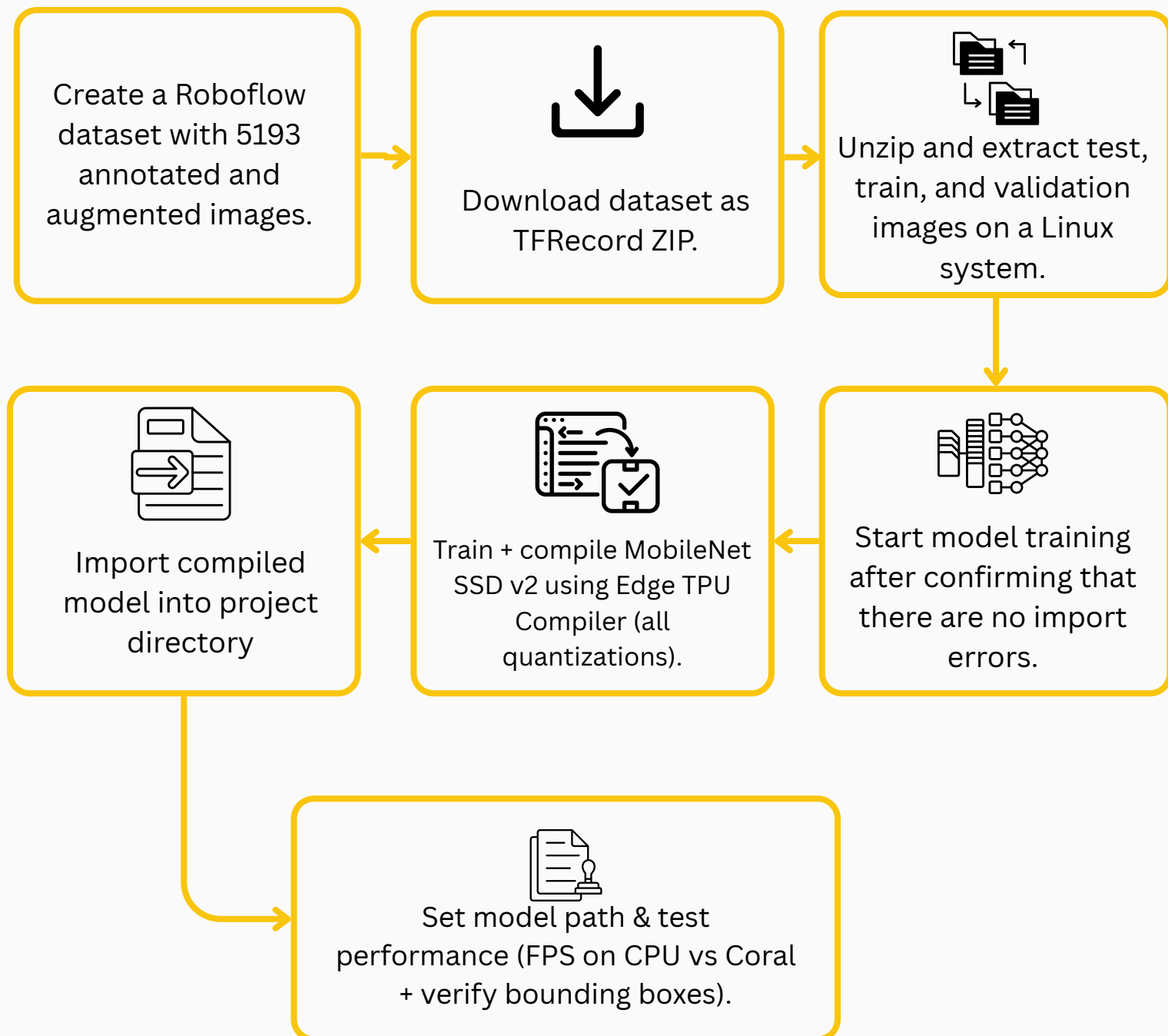
```
PROCEDURE DriveRoutine
  WHILE ButtonIsPressed()
    CALL UpdateSensors()
    IF lap_finish = FALSE THEN
      IF ParkingDetected() THEN
        inParkingAtStart ← TRUE
        IF TF_Left < 20 AND TF_Head < 200 THEN
          RightParking()
        ELSEIF TF_Right < 20 AND TF_Head < 200 THEN
          LeftParking()
        ENDIF
      ENDIF
      CALL DriveLogic()
    ELSE
      CALL ParkingLogic()
    ENDIF
  ENDWHILE
ENDPROCEDURE
```

FLOWCHART

OBSTACLE CHALLENGE



OBJECT DETECTION MODEL



FUTURE ENGINEERS



ANALYTICS



FUTURE ENGINEERS



APPROACH

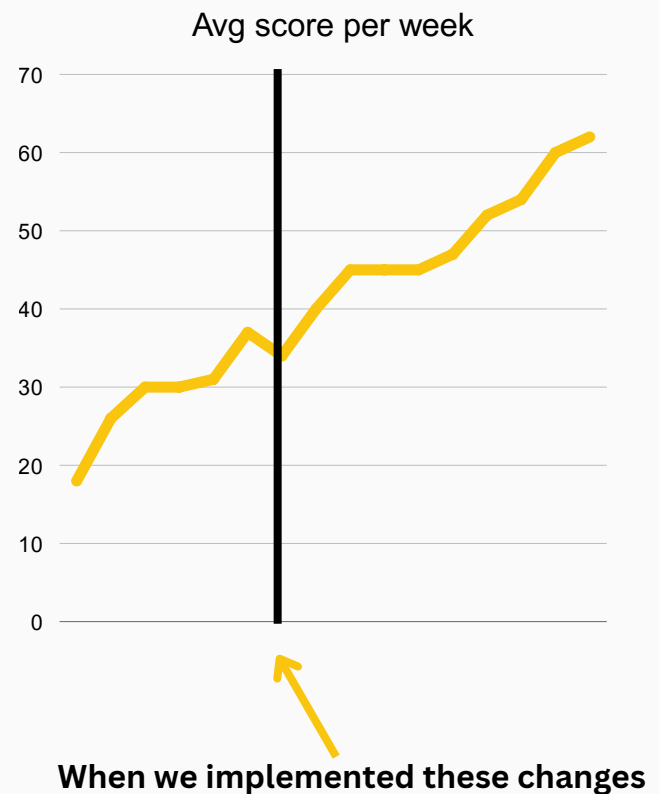
The approach and prototyping that was done during the season to get the most out of the robot

Adoption of Lane Switching Logic:

- Improved Navigation: Implementing lane switching allowed the robot to effectively avoid obstacles and navigate more smoothly.
- Simpler Implementation: Lane switching proved to be more straightforward to implement and integrate with the AI model, compared to traditional turning methods.
- Enhanced Reliability: This approach resulted in more consistent and reliable performance, reducing errors and increasing the robot's efficiency.

Transition from Colour Sensor to Camera with AI Model:

- Increased Accuracy: The adoption of an AI model significantly enhanced the robot's accuracy in detecting and navigating the course.
- Light Condition Robustness: Unlike traditional colour sensors, the camera and AI model were less affected by changes in lighting, ensuring consistent performance.
- Speed and Efficiency: The AI model facilitated quicker decision-making, reducing response times and boosting overall performance.



FUTURE ENGINEERS



PERFORMANCE

Before nationals

Trial	Sections	Laps	Stop at finish	Hit block	Start with park	Park	Total
1	24	3	3	8	7	7	52
2	24	3	3	10	7	15	62
3	24	3	3	8	7	15	60

After nationals

Trial	Sections	Laps	Stop at finish	Hit block	Start with park	Park	Total
1	24	3	3	10	7	15	62
2	24	3	3	8	7	15	60
3	24	3	3	10	7	15	62

SOCIALS



FUTURE ENGINEERS



YOUTUBE CHANNEL



FUTURE ENGINEERS



GITHUB



FUTURE ENGINEERS

